Environment Semantics

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss the difference between **dynamic** and **lexical** scoping
- Introduce **closures** as a way of implementing lexical scoping in the environment model
- Discuss **recursion** and closures

Variable Scoping

Two Major Concerns

Two Major Concerns

1. Are variables *mutable*?

Two Major Concerns

1. Are variables *mutable*?
2. How are variables *scoped*? (Dynamically or lexically?)

Two Major Concerns

1. Are variables *mutable*?

2. How are variables *scoped*? (Dynamically or lexically?)

OCaml variables are immutable, binding defined, and lexically scoped

Mutability

```
let x = 0
let f () =
  let x = 1 in
  ()
print_int x
```

Immutable (OCaml)

```
x = 0
def f():
    global x
    x = 1
print(x)
```

Mutable (Python)

Mutability

```
let x = 0
let f () =
  let x = 1 in
  ()
print_int x
```

Immutable (OCaml)

```
x = 0
def f():
    global x
    x = 1
print(x)
```

Mutable (Python)

Def. *(informal)* A variable is **mutable** if we are allowed to change its value after it has been declared

Scope

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

There are two standard paradigms: **dynamic** scoping and **lexical** scoping (static scoping)

Scope

Def. *(Informal)* The **scope** of a variable binding is *when* and *where* a variable can be accessed

There are two standard paradigms: **dynamic** scoping and **lexical** scoping (static scoping)

Warning. Scope is one of the most overused terms in computer science...

Dynamic Scoping

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic Scoping

```bash
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic scoping refers to when bindings are determined at runtime based on *computational context*

Dynamic Scoping

```bash
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

Bash

Dynamic scoping refers to when bindings are determined at runtime based on *computational context*

This is a *temporal view*, i.e., what a computation done beforehand which affected the value of a variable

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

- The binding defines it's own scope (let-bindings)

Lexical Scoping

```python
x = 0
def f():
    x = 1
    return x
assert(f() == 1)
assert(x == 0)
```

Python

```ocaml
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

OCaml

Lexical (static) scoping refers to the use of textual delimiters to define the scope of a binding

There are two common ways lexical scope is determined:

- The binding defines it's own scope (let-bindings)
- A block defines the scope of a variable (python functions)

Tradeoffs

```
f() { x=23; g; }
g() { y=$x; }
f
echo $y
```

dynamic

vs.

```
let x = 0
let f () =
  let x = 1 in
  x
let _ = assert (f () = 1)
let _ = assert (x = 0)
```

lexical

Implementing dynamic scoping is *way* easier...

But **every modern PL** implements lexical scoping

The Environment Model

Motivation

Environment Semantic Judgments

Environments

Syntax (λ Calculus +)

Environment Semantics (λ Calculus +)

Back to Scoping

$\langle\ \{x \mapsto \text{true}\ , f \mapsto \lambda y\,.\,x\}, \text{let}\ x = \text{false}\ \text{in}\ f\ x\ \rangle$

Closures

Closures

Values (λ Calculus +)

Semantics Again (λ Calculus +)

Example

$\langle\ \{x \mapsto \text{true}\}\ ,\ \text{let}\ f = \lambda y\,.\,x\ \text{in let}\ x = \text{false in}\ f\ x\ \rangle$

Recursion

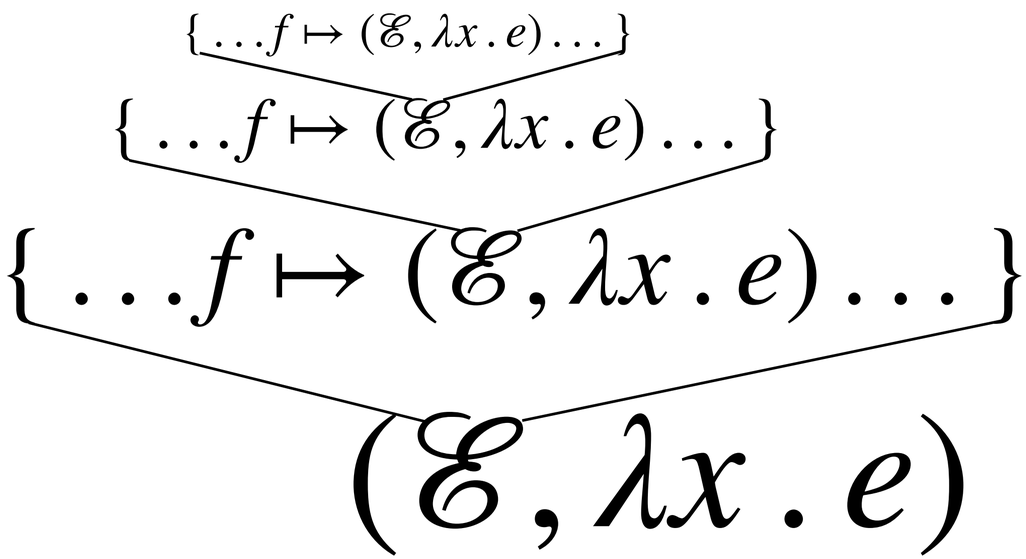
High-Level

```
let f x =
  if x = 0
  then 1
  else f (x - 1)
in f 10
```

We've only considered *non-recursive* function so far

And there's no easy way to do recursion with substitution

The Problem



In order to implement recursion, a closure has to "*know thyself*"

But we **can't** build infinite structures like this in OCaml

Named Closures

Lambda Calculus^{++} (Syntax, Again)

Semantics Again Again (λ Calculus +)

demo